Probabilistic Machine Learning

Block 2 – Probabilistic Classification

Andrés Masegosa

Aalborg Unviersity

2026

Outline

1. Probabilistic Classification
2. Extension: Neural Probabilistic Classifier

The Same Recipe, New Problem

In Block 1 we applied the probabilistic recipe to **regression**:

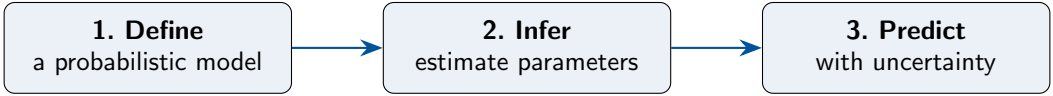
$$y \mid x \sim \mathcal{N}(f_\theta(x),\ \sigma^2)$$

The Same Recipe, New Problem

In Block 1 we applied the probabilistic recipe to **regression**:

$$y \mid x \sim \mathcal{N}(f_\theta(x),\ \sigma^2)$$

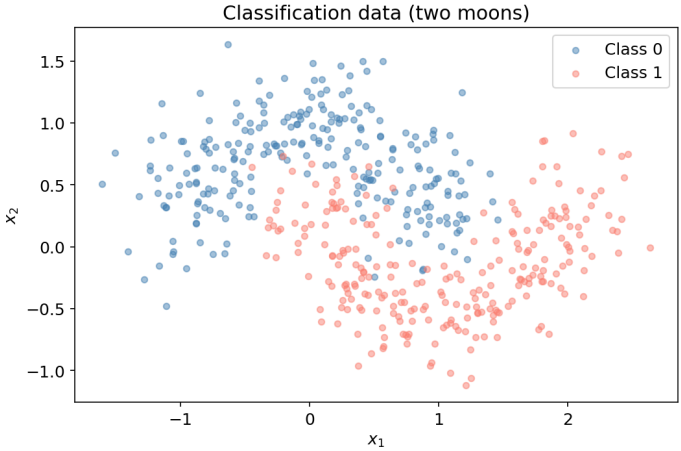
Now we apply **the same recipe** to **classification**:



What changes is the **distribution**: from Gaussian to Bernoulli.

Classification: The Setup

Given pairs $\{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ with $y_i \in \{0, 1\}$.



The Probabilistic Model: Bernoulli

For binary classification, the natural distribution is **Bernoulli**:

$$\boxed{y \mid \mathbf{x} \sim \mathrm{Bernoulli}\big(\sigma(\mathbf{w}^\top \mathbf{x} + b)\big)}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

The Probabilistic Model: Bernoulli

For binary classification, the natural distribution is **Bernoulli**:

$$\boxed{y \mid \mathbf{x} \sim \mathrm{Bernoulli}\big(\sigma(\mathbf{w}^\top \mathbf{x} + b)\big)}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the **sigmoid function**.

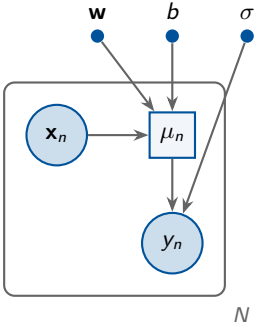
This is exactly **logistic regression** — but now we see where it comes from:

- The output is a probability $p \in [0, 1]$.
- The sigmoid maps any real number to $[0, 1]$.
- The Bernoulli models binary outcomes given that probability.

Graphical Model: Regression vs. Classification

Regression (Block 1)

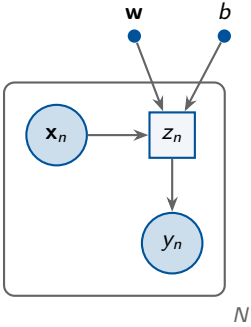
$$y_n \sim \mathcal{N}(\mu_n, \sigma^2)$$



$\Rightarrow$

Classification (Block 2)

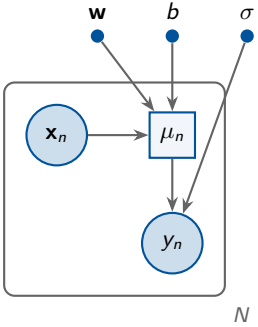
$$y_n \sim \text{Bernoulli}(\sigma(z_n))$$



Graphical Model: Regression vs. Classification

Regression (Block 1)

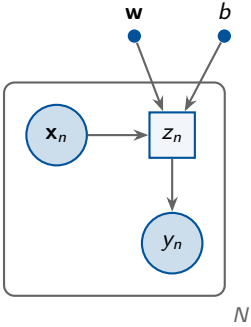
$$y_n \sim \mathcal{N}(\mu_n, \sigma^2)$$



$\Rightarrow$

Classification (Block 2)

$$y_n \sim \text{Bernoulli}(\sigma(z_n))$$



Same structure: replace Gaussian with Bernoulli, drop σ.
The plate (`pyro.plate`) and parameter nodes (`pyro.param`) are identical.

Likelihood and Cross-Entropy

The likelihood for Bernoulli observations:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{N} p_i^{y_i}(1 - p_i)^{1-y_i} \qquad \text{where } p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$$

Likelihood and Cross-Entropy

The likelihood for Bernoulli observations:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{N} p_i^{y_i} (1 - p_i)^{1-y_i} \qquad \text{where } p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$$

The negative log-likelihood:

$$-\log p(\mathcal{D} \mid \theta) = -\sum_{i=1}^{N} \big[\, y_i \log p_i + (1 - y_i) \log(1 - p_i) \,\big]$$

Likelihood and Cross-Entropy

The likelihood for Bernoulli observations:

$$p(\mathcal{D} \mid \theta) = \prod_{i=1}^{N} p_i^{y_i}(1-p_i)^{1-y_i} \qquad \text{where } p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$$

The negative log-likelihood:

$$-\log p(\mathcal{D} \mid \theta) = -\sum_{i=1}^{N} \big[\, y_i \log p_i + (1-y_i)\log(1-p_i) \,\big]$$

Key insight

This is the **binary cross-entropy** loss.
Just as MSE came from Gaussian noise, cross-entropy comes from a Bernoulli model.

What Does Probability Give Us?

A standard classifier returns a **class label**: $\hat{y} = 0$ or $\hat{y} = 1$.

A probabilistic classifier returns a **probability**: $p(y = 1 \mid \mathbf{x})$.

What Does Probability Give Us?

A standard classifier returns a **class label**: $\hat{y} = 0$ or $\hat{y} = 1$.

A probabilistic classifier returns a **probability**: $p(y = 1 \mid \mathbf{x})$.

This lets us distinguish:

Confident prediction

$p(y = 1 \mid \mathbf{x}) = 0.98$

The model is "sure" — act on it.

Uncertain prediction

$p(y = 1 \mid \mathbf{x}) = 0.55$

Almost a coin flip — be cautious.

Both would be classified as $\hat{y} = 1$, but they carry very different information.

Why Calibration Matters

Imagine an AI tool that assists **medical diagnosis**:

"The model says there is a **95% probability** *of malignancy."*

Why Calibration Matters

Imagine an AI tool that assists **medical diagnosis**:

"The model says there is a **95% probability** *of malignancy."*

If the model is well-calibrated, the doctor trusts this and recommends surgery.

But if the model is **poorly calibrated**, that 95% might really mean 70%.

Why Calibration Matters

Imagine an AI tool that assists **medical diagnosis**:

"The model says there is a **95% probability** *of malignancy."*

If the model is well-calibrated, the doctor trusts this and recommends surgery.

But if the model is **poorly calibrated**, that 95% might really mean 70%.

The stakes

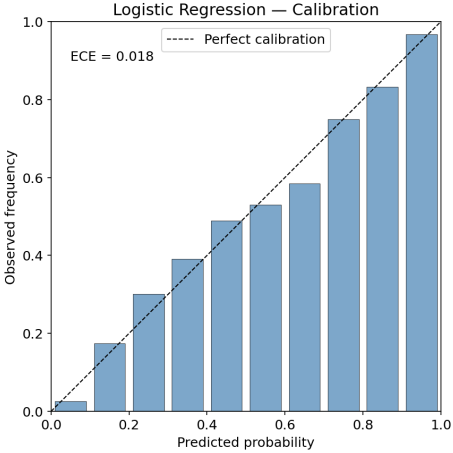
At 95% $\rightarrow$ surgery. At 70% $\rightarrow$ further tests first.

Same prediction, very different decisions. The number must be *trustworthy*, not just high.

Calibration

A model is **well-calibrated** if its predicted probabilities match observed frequencies.

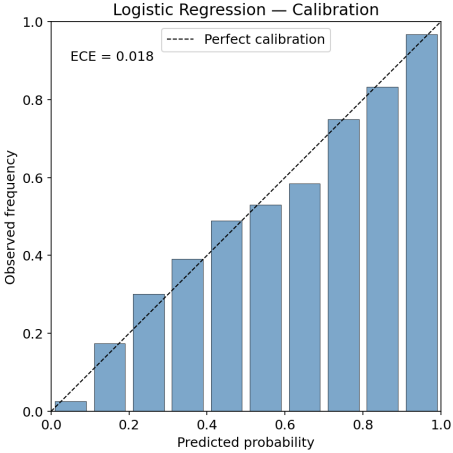
"When the model says 0.7, it should be right about 70% of the time."



Calibration

A model is **well-calibrated** if its predicted probabilities match observed frequencies.

"When the model says 0.7, it should be right about 70% of the time."



Measuring Calibration: ECE

We measure calibration with the **Expected Calibration Error** (ECE):

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left| \, \mathrm{acc}(b) - \mathrm{conf}(b) \, \right|$$

where we partition predictions into B bins by confidence, and:

- $\mathrm{conf}(b)$ = average predicted probability in bin b
- $\mathrm{acc}(b)$ = fraction of correct predictions in bin b
- n_b = number of predictions in bin b

Measuring Calibration: ECE

We measure calibration with the **Expected Calibration Error** (ECE):

$$\mathrm{ECE} = \sum_{b=1}^{B} \frac{n_b}{N} \left| \, \mathrm{acc}(b) - \mathrm{conf}(b) \, \right|$$

where we partition predictions into B bins by confidence, and:

- $\mathrm{conf}(b)$ = average predicted probability in bin b
- $\mathrm{acc}(b)$ = fraction of correct predictions in bin b
- n_b = number of predictions in bin b

Interpretation:

- $\mathrm{ECE} = 0 \Rightarrow$ perfectly calibrated.
- ECE is the **weighted average gap** between the bars and the diagonal in the reliability diagram.

Summary: Regression vs. Classification

	Regression (Block 1)	**Classification (Block 2)**
Output	continuous $y \in \mathbb{R}$	binary $y \in \{0, 1\}$
Distribution	Gaussian	Bernoulli
Model	$\mathcal{N}(f_\theta(x), \sigma^2)$	$\mathrm{Bernoulli}(\sigma(f_\theta(x)))$
NLL =	MSE	Cross-entropy
Uncertainty	$\pm\sigma$ bands	predicted probability

Summary: Regression vs. Classification

	Regression (Block 1)	**Classification (Block 2)**
Output	continuous $y \in \mathbb{R}$	binary $y \in \{0, 1\}$
Distribution	Gaussian	Bernoulli
Model	$\mathcal{N}(f_\theta(x), \sigma^2)$	$\mathrm{Bernoulli}(\sigma(f_\theta(x)))$
NLL =	MSE	Cross-entropy
Uncertainty	$\pm\sigma$ bands	predicted probability

Same framework. Only the distribution changes.

Let's See It in Code

Time to open the **Pyro notebook**!

What we will implement:

1. Logistic regression as a probabilistic model in Pyro.
2. Train via MLE (same SVI setup as Block 1).
3. Analyze predictions: confident vs. uncertain examples.
4. Calibration analysis.

From Logistic Regression to Neural Networks

Same idea as in Block 1 — replace the linear function:

Logistic regression:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

From Logistic Regression to Neural Networks

Same idea as in Block 1 — replace the linear function:

Logistic regression:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Neural classifier:

$$p(y = 1 \mid \mathbf{x}) = \sigma(f_\theta(\mathbf{x})) \qquad \text{where } f_\theta = \text{neural network}$$

From Logistic Regression to Neural Networks

Same idea as in Block 1 — replace the linear function:

Logistic regression:

$$p(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$$

Neural classifier:

$$p(y = 1 \mid \mathbf{x}) = \sigma(f_\theta(\mathbf{x})) \qquad \text{where } f_\theta = \text{neural network}$$

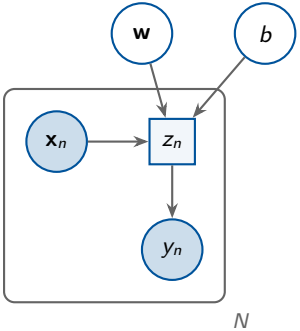
For multi-class (K classes), the sigmoid becomes **softmax**:

$$p(y = k \mid \mathbf{x}) = \frac{e^{f_\theta(\mathbf{x})_k}}{\sum_{j=1}^{K} e^{f_\theta(\mathbf{x})_j}}$$

And Bernoulli becomes **Categorical**.

Same Graphical Model, Different Function

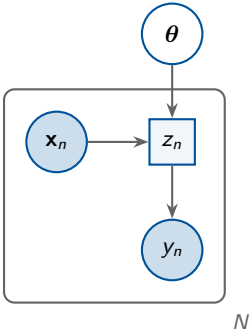
Logistic regression



$$z_n = \mathbf{w}^\top \mathbf{x}_n + b$$

$\Rightarrow$

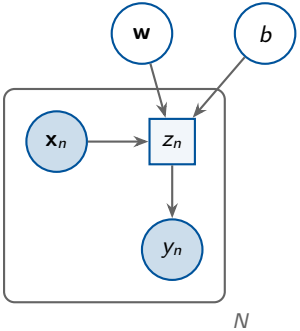
Neural classifier



$$z_n = f_\theta(\mathbf{x}_n)$$

Same Graphical Model, Different Function

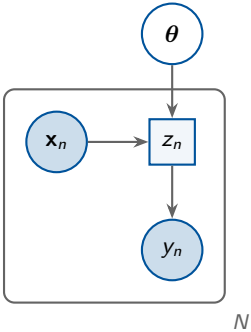
Logistic regression



$z_n = \mathbf{w}^\top \mathbf{x}_n + b$

$\Rightarrow$

Neural classifier



$z_n = f_\theta(\mathbf{x}_n)$

Identical graphical model structure. The probabilistic layer ($y_n \sim \mathrm{Bernoulli}(\sigma(z_n))$) is the same — only the deterministic computation of z_n changes. Same Pyro code, different `nn.Module`.

More Capacity $\neq$ Better Calibration

Neural networks are powerful function approximators, but:

- Modern deep networks tend to be **overconfident**.
- A model that says $p = 0.99$ might only be right 80% of the time.
- More parameters and capacity can **worsen** calibration.

More Capacity $\neq$ Better Calibration

Neural networks are powerful function approximators, but:

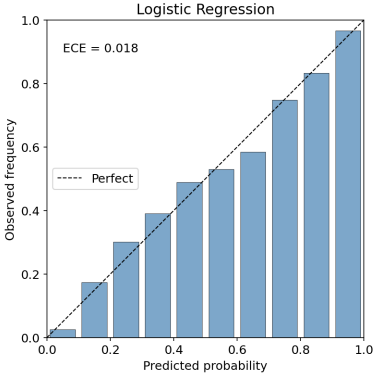
- Modern deep networks tend to be **overconfident**.
- A model that says $p = 0.99$ might only be right 80% of the time.
- More parameters and capacity can **worsen** calibration.

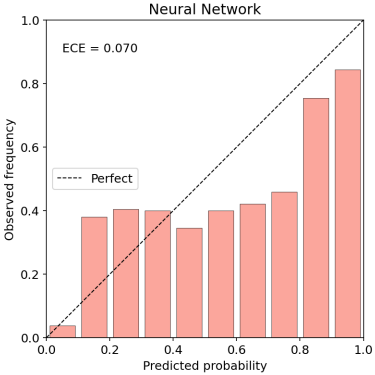
This is why the probabilistic perspective matters even with powerful models:

Getting a probability is easy (just use softmax).
Getting a *reliable* probability requires careful modeling.

More Capacity $\neq$ Better Calibration

Neural networks are powerful function approximators, but:





Fixing Calibration: Temperature Scaling

The **simplest** post-hoc calibration method: learn a single parameter T:

$$p_{\text{calibrated}} = \sigma\left(\frac{z}{T}\right) \qquad \text{where } z = f_\theta(\mathbf{x}) \text{ are the logits}$$

Fixing Calibration: Temperature Scaling

The **simplest** post-hoc calibration method: learn a single parameter T:

$$p_{\text{calibrated}} = \sigma\left(\frac{z}{T}\right) \qquad \text{where } z = f_\theta(\mathbf{x}) \text{ are the logits}$$

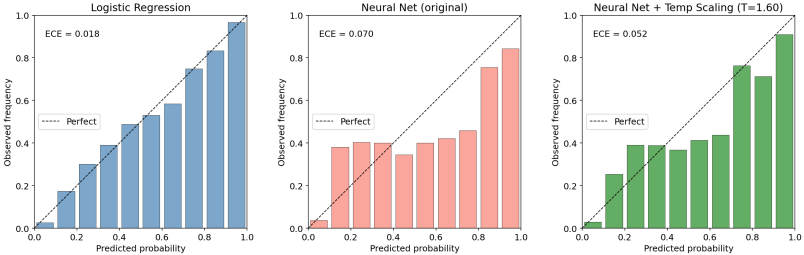
- $T > 1 \Rightarrow$ **softens** overconfident predictions.
- $T < 1 \Rightarrow$ sharpens predictions.
- $T = 1 \Rightarrow$ original model (no change).
- Fit T on a **validation set** by minimizing NLL. That's it.

Fixing Calibration: Temperature Scaling

The **simplest** post-hoc calibration method: learn a single parameter T:

$$p_{\text{calibrated}} = \sigma\left(\frac{z}{T}\right) \qquad \text{where } z = f_\theta(\mathbf{x}) \text{ are the logits}$$

- $T > 1 \Rightarrow$ **softens** overconfident predictions.
- $T < 1 \Rightarrow$ sharpens predictions.
- $T = 1 \Rightarrow$ original model (no change).
- Fit T on a **validation set** by minimizing NLL. That's it.



The Calibration Landscape

Temperature scaling is the **simplest fix** — but calibration is an active research area:

- **Post-hoc methods:** temperature scaling, Platt scaling, isotonic regression, histogram binning...
- **Training-time methods:** label smoothing, mixup, focal loss...
- **Bayesian methods:** put distributions over parameters, capturing **epistemic uncertainty** — this addresses overconfidence *at the source* rather than patching it afterwards.

The Calibration Landscape

Temperature scaling is the **simplest fix** — but calibration is an active research area:

- **Post-hoc methods:** temperature scaling, Platt scaling, isotonic regression, histogram binning...
- **Training-time methods:** label smoothing, mixup, focal loss...
- **Bayesian methods:** put distributions over parameters, capturing **epistemic uncertainty** — this addresses overconfidence *at the source* rather than patching it afterwards.

Day 2 preview: Bayesian inference is one of the most principled ways to improve calibration — because an uncertain model naturally produces less extreme probabilities.

Let's Compare in Code

Notebook: **Neural Probabilistic Classifier**

What we will implement:

1. Neural network classifier in Pyro (same structure as before).
2. Compare predictions with the logistic model.
3. Compare **calibration**: is the neural net more reliable?
4. Apply **temperature scaling** and see the improvement.

Same Pyro code structure — only the predictor changes.